

Branch: MCA (Data Science) Kargil	Semester: 2
Student Name: Chaitanya Sharma	UID: 25MCD10056
Subject Name: Technical Training - I Lab	Subject Code: 25CAP-652
Section/Group: 25MCD KAR-1	Date of Performance: 31-03-2026

Experiment 8

Experiment Aim:

Implementation of Joins in PostgreSQL to retrieve and combine data from multiple related tables. (Company Tags: Oracle, SAP, IBM, Microsoft)

Tools used: PostgreSQL

Objectives:

- **Join Operations:** To understand how different types of joins (INNER, LEFT, RIGHT, CROSS) are used to combine data from multiple relations.
- **Data Retrieval:** To learn how to extract meaningful information from relational schemas using join conditions.
- **Handling Missing Data:** To identify and manage NULL values in join operations.
- **Query Optimization:** To write efficient queries for real-world database scenarios.

Theory:

A JOIN in SQL is used to combine rows from two or more tables based on a related column between them. Joins are fundamental in relational databases as they allow retrieval of distributed data efficiently.

1. INNER JOIN: Returns only matching records from both tables.
2. LEFT JOIN: Returns all records from the left table and matching records from the right table.
3. RIGHT JOIN: Returns all records from the right table and matching records from the left table.
4. CROSS JOIN: Returns the Cartesian product of both tables.
5. MULTIPLE JOIN: Combines more than two tables using join conditions.

Experiment Steps:

Step 1: Creating Base Tables

Query:

-- Create Employees table

CREATE TABLE Employees (

emp_id INT PRIMARY KEY,

name VARCHAR(50),

dept_id INT

);

SELECT * FROM departments;

-- Create Projects table

CREATE TABLE Projects (

project_id INT PRIMARY KEY,

project_name VARCHAR(50)

);

-- Create Assignments table (like enrollments)

CREATE TABLE Assignments (

emp_id INT,

project_id INT,

PRIMARY KEY (emp_id, project_id)

);

-- Create Departments table

CREATE TABLE Departments (

dept_id INT PRIMARY KEY,

dept_name VARCHAR(50)

);

Step 2: Inserting Sample Data

Query:

-- Employees

INSERT INTO Employees VALUES

(1, 'Rahul', 101),

(2, 'Priya', 102),

(3, 'Amit', 101),

(4, 'Neha', 103);

	emp_id [PK] integer	name character varying (50)	dept_id integer
1	1	Rahul	101
2	2	Priya	102
3	3	Amit	101
4	4	Neha	103

-- Projects

INSERT INTO Projects VALUES

(201, 'ERP System'),

(202, 'AI Chatbot'),

(203, 'Data Analytics');

	project_id [PK] integer	project_name character varying (50)
1	201	ERP System
2	202	AI Chatbot
3	203	Data Analytics

-- Assignments

INSERT INTO Assignments VALUES

(1, 201),

(1, 202),

(2, 202),

(3, 203);

	emp_id [PK] integer	project_id [PK] integer
1	1	201
2	1	202
3	2	202
4	3	203

-- Departments

INSERT INTO Departments VALUES

(101, 'Engineering'),

(102, 'IT'),

(103, 'Operations');

	dept_id [PK] integer	dept_name character varying (50)
1	101	Engineering
2	102	IT
3	103	Operations

Step 3: INNER JOIN

SELECT e.name AS employee_name, p.project_name

FROM Employees e

INNER JOIN Assignments a

ON e.emp_id = a.emp_id

INNER JOIN Projects p

ON a.project_id = p.project_id;

	employee_name character varying (50)	project_name character varying
1	Rahul	ERP System
2	Rahul	AI Chatbot
3	Priya	AI Chatbot
4	Amit	Data Analytics

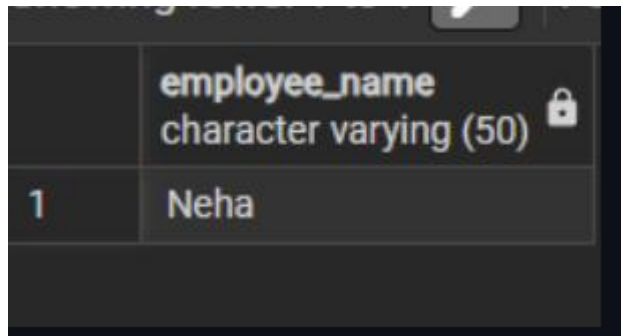
Step 4: LEFT JOIN

Query:

-- Left join:

```
SELECT e.name AS employee_name  
FROM Employees e  
LEFT JOIN Assignments a  
    ON e.emp_id = a.emp_id  
WHERE a.project_id IS NULL;
```

Output:



	employee_name character varying (50)
1	Neha

Step 5: RIGHT JOIN

Query:

-- Right join:

```
SELECT p.project_name, e.name AS employee_name
FROM Employees e
RIGHT JOIN Assignments a
    ON e.emp_id = a.emp_id
RIGHT JOIN Projects p
    ON a.project_id = p.project_id;
```

Output:

	project_name character varying (50) 🔒	employee_name character varying (50) 🔒
1	ERP System	Rahul
2	AI Chatbot	Rahul
3	AI Chatbot	Priya
4	Data Analytics	Amit

Step 6: MULTIPLE JOIN (Students with Department Info)

Query:

-- Multiple join:

```
SELECT e.name AS employee_name, d.dept_name
FROM Employees e
```


JOIN Departments d

ON e.dept_id = d.dept_id;

Output:

	employee_name character varying (50) 🔒	dept_name character varying (50) 🔒
1	Rahul	Engineering
2	Priya	IT
3	Amit	Engineering
4	Neha	Operations

Step 7: CROSS JOIN

Query:

-- Cross join:

```
SELECT e.name AS employee_name, p.project_name
FROM Employees e
CROSS JOIN Projects p;
```

Output:

employee_name character varying (50) 🔒	project_name character varying (50) 🔒
Rahul	ERP System
Priya	ERP System
Amit	ERP System
Neha	ERP System
Rahul	AI Chatbot
Priya	AI Chatbot
Amit	AI Chatbot
Neha	AI Chatbot
Rahul	Data Analytics
Priya	Data Analytics
Amit	Data Analytics
Neha	Data Analytics

Outcomes:

- I will be able to implement different types of joins in SQL.
- I will understand how to retrieve and combine related data from multiple tables.

- I will gain hands-on experience in handling NULL values and join conditions.
- I will be able to apply join operations in real-world database scenarios.